

PZ-OV7670 摄像头模块开发手册

本手册我们将向大家介绍 PZ-OV7670 摄像头模块及其使用。本手册我们将在普中玄武&朱雀开发板上驱动 PZ-OV7670 摄像头模块，实现摄像头功能。本章分为如下几部分内容：

- 1 OV7670 介绍
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 OV7670 介绍

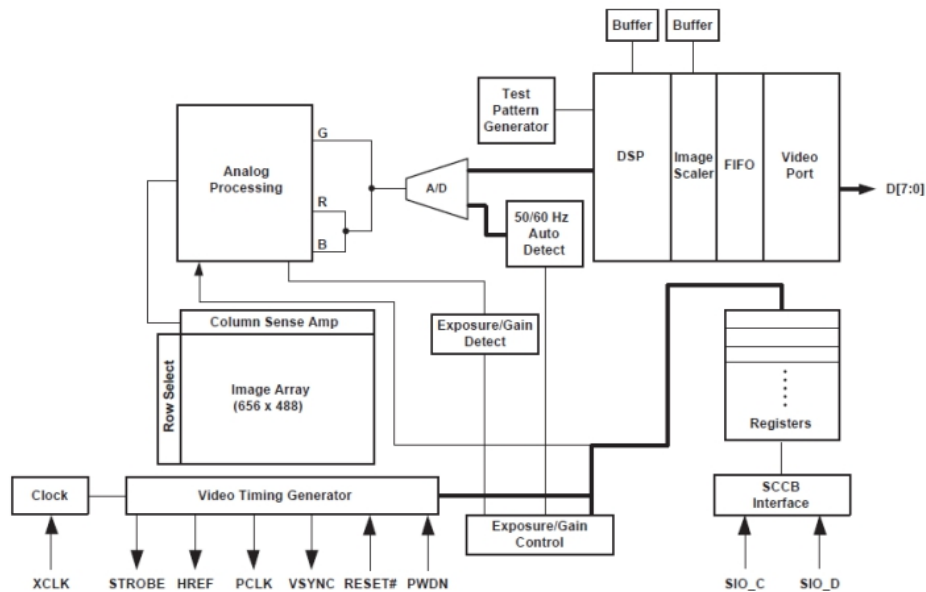
1.1 OV7670 简介

OV7670 是 OV (OmniVision) 公司生产的一颗 1/6 寸的 CMOS VGA 图像传感器。该传感器体积小、工作电压低，提供单片 VGA 摄像头和影像处理器的所有功能。通过 SCCB 总线控制，可以输出整帧、子采样、取窗口等方式的各种分辨率 8 位影像数据。该产品 VGA 图像最高达到 30 帧/秒。用户可以完全控制图像质量、数据格式和传输方式。所有图像处理功能过程包括伽玛曲线、白平衡、度、色度等都可以通过 SCCB 接口编程。OmniVision 图像传感器应用独有的传感器技术，通过减少或消除光学或电子缺陷如固定图案噪声、托尾、浮散等，提高图像质量，得到清晰的稳定的彩色图像。

OV7670 的特点有：

- (1) 高灵敏度、低电压适合嵌入式应用
- (2) 标准的 SCCB 接口，兼容 IIC 接口
- (3) 支持 RawRGB、RGB(GBR4:2:2, RGB565/RGB555/RGB444), YUV(4:2:2) 和 YCbCr (4:2:2) 输出格式
- (4) 支持 VGA、CIF, 和从 CIF 到 40*30 的各种尺寸输出
- (5) 支持自动曝光控制、自动增益控制、自动白平衡、自动消除灯光条纹、自动黑电平校准等自动控制功能。同时支持色饱和度、色相、伽马、锐度等设置。
- (6) 支持闪光灯
- (7) 支持图像缩放

OV7670 的功能框图图如图所示：



OV7670 传感器包括如下一些功能模块：

1. 感光阵列（ Image Array）

OV7670 总共有 656*488 个像素，其中 640*480 个有效（即有效像素为 30W）。

2. 时序发生器（ Video Timing Generator）

时序发生器具有的功能包括：阵列控制和帧率发生（ 7 种不同格式输出）、内部信号发生器和分布、帧率时序、自动曝光控制、输出外部时序（ VSYNC、HREF/HSYNC 和 PCLK）。

3. 模拟信号处理（ Analog Processing）

模拟信号处理所有模拟功能，并包括：自动增益（ AGC）和自动白平衡（ AWB）。

4. A/D 转换（ A/D）

原始的信号经过模拟处理器模块之后，分 G 和 BR 两路进入一个 10 位的 A/D 转换器，A/D 转换器工作在 12M 频率，与像素频率完全同步（转换的频率和帧率有关）。

除 A/D 转换器外，该模块还有以下三个功能：

- ①黑电平校正（ BLC）
- ②U/V 通道延迟
- ③A/D 范围控制

A/D 范围乘积和 A/D 的范围控制共同设置 A/D 的范围和最大值，允许用户

根据应用调整图片的亮度。

5. 测试图案发生器 (Test Pattern Generator)

测试图案发生器功能包括：八色彩色条图案、渐变至黑白彩色条图案和输出脚移位“1”。

6. 数字处理器 (DSP)

这个部分控制由原始信号插值到 RGB 信号的过程，并控制一些图像质量：

- ①边缘锐化（二维高通滤波器）
- ②颜色空间转换（原始信号到 RGB 或者 YUV/YCbYCr）
- ③RGB 色彩矩阵以消除串扰
- ④色相和饱和度的控制
- ⑤黑/白点补偿
- ⑥降噪
- ⑦镜头补偿
- ⑧可编程的伽玛
- ⑨十位到八位数据转换

7. 缩放功能 (Image Scaler)

这个模块按照预先设置的要求输出数据格式，能将 YUV/RGB 信号从 VGA 缩小到 CIF 以下的任何尺寸。

8. 数字视频接口 (Digital Video Port)

通过寄存器 COM2[1:0]，调节 IOL/IOH 的驱动电流，以适应用户的负载。

9. SCCB 接口 (SCCB Interface)

SCCB 接口控制图像传感器芯片的运行，详细使用方法参照《OmniVisionTechnologies Seril Camera Control Bus(SCCB) Specification》这个文档。

10. LED 和闪光灯的输出控制 (LED and Storbe Flash Control Output)

OV7670 有闪光灯模式，可以控制外接闪光灯或闪光 LED 的工作。

OV7670 的寄存器通过 SCCB 时序访问并设置，SCCB 时序和 IIC 时序十分类似，在这里我们不做介绍，请大家参考模块的相关文档。

接下来我们介绍一下 OV7670 的图像数据输出格式。首先我们简单介绍几个

定义：

VGA，即分辨率为 640*480 的输出模式；

QVGA，即分辨率为 320*240 的输出格式，也就是本手册我们需要用到的格式；

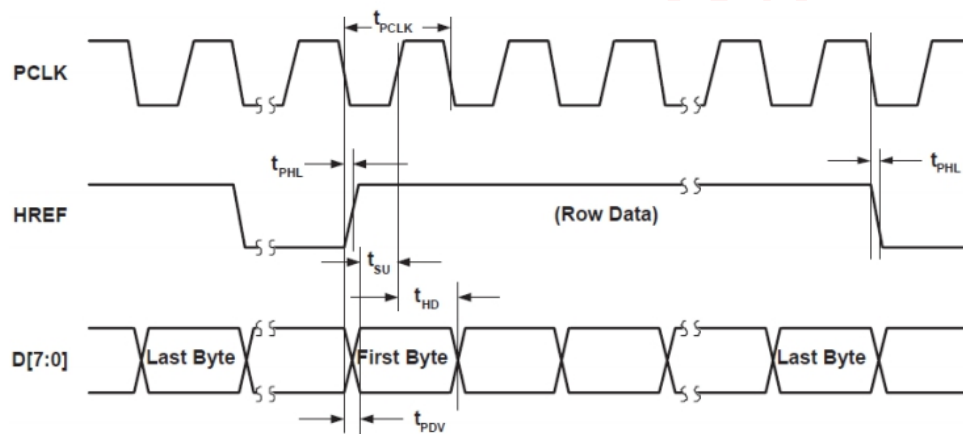
QQVGA，即分辨率为 160*120 的输出格式；

PCLK，即像素时钟，一个 PCLK 时钟，输出一个像素(或半个像素)。

VSYNC，即帧同步信号。

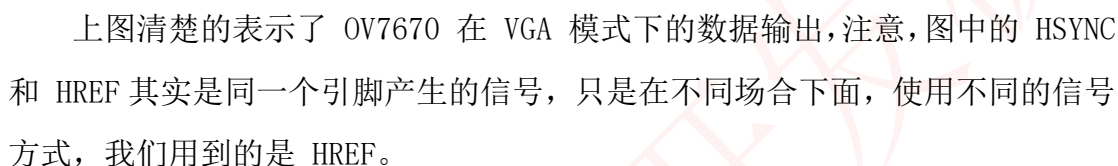
HREF /HSYNC，即行同步信号。

OV7670 的图像数据输出（通过 D[7:0]）就是在 PCLK，VSYNC 和 HREF/HSYNC 的控制下进行的。首先看看行输出时序，如图所示：

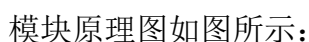


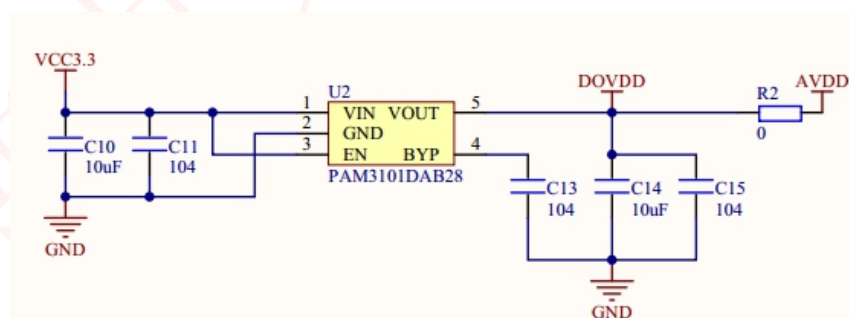
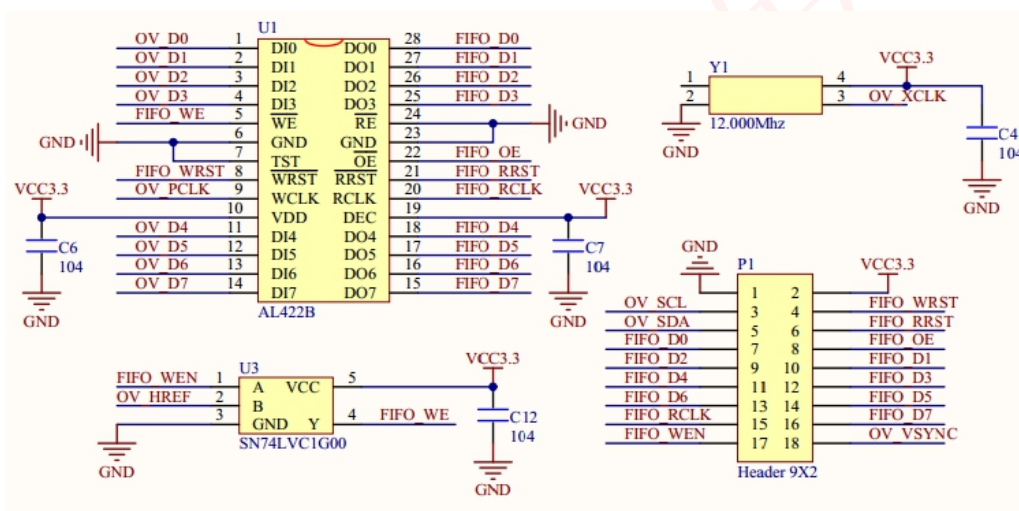
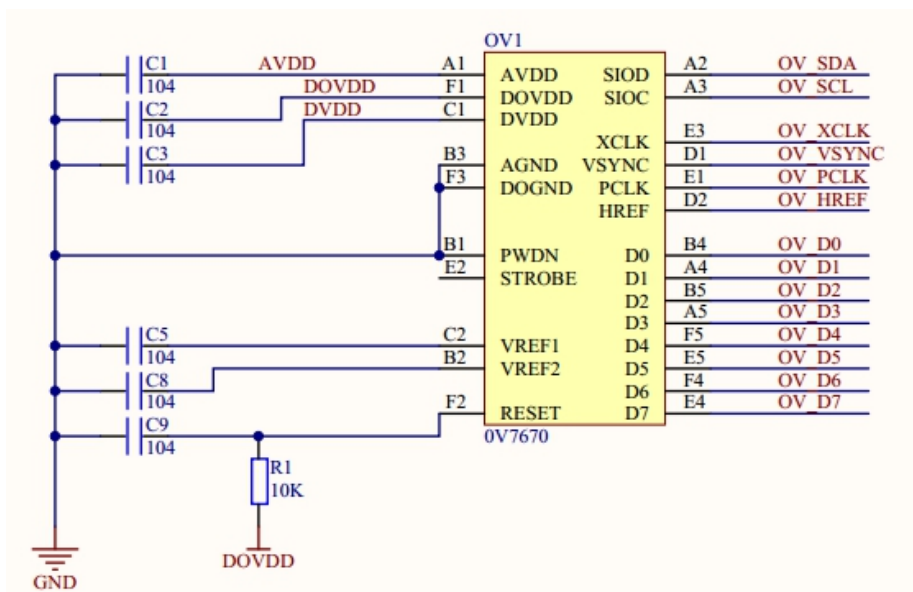
从上图可以看出，图像数据在 HREF 为高的时候输出，当 HREF 变高后，每一个 PCLK 时钟，输出一个字节数据。比如我们采用 VGA 时序，RGB565 格式输出，每 2 个字节组成一个像素的颜色（高字节在前，低字节在后），这样每行输出总共有 640*2 个 PCLK 周期，输出 640*2 个字节。

再看看帧时序（VGA 模式），如图所示：



接下来我们介绍一下 PZ-0V7670 摄像头模块。该模块的外观如图所示:





从上图可以看出，PZ-OV7670 摄像头模块自带了有源晶振 Y1，用于产生 12M 时钟作为 OV7670 的 XCLK 输入。同时自带了稳压芯片，用于提供 OV7670 稳定的 2.8V 工作电压，并带有一个 FIFO 芯片 (AL422B)，该 FIFO 芯片的容量是 384K 字节，足够存储 2 帧 QVGA 的图像数据。模块通过一个 2*9 的双排排针 (P1) 与外部通信，该管脚功能如图所示：

信号	作用描述	信号	作用描述
VCC3.3	模块供电脚，接 3.3V 电源	FIFO_WEN	FIFO 写使能
GND	模块地线	FIFO_WRST	FIFO 写指针复位
OV_SCL	SCCB 通信时钟信号	FIFO_RRST	FIFO 读指针复位
OV_SDA	SCCB 通信数据信号	FIFO_OE	FIFO 输出使能（片选）
FIFO_D[7:0]	FIFO 输出数据（8 位）	OV_VSYNC	OV7670 帧同步信号
FIFO_RCLK	读 FIFO 时钟		

1.2 PZ-OV7670 模块使用方法

下面我们来看看如何使用 PZ-OV7670 摄像头模块（以 QVGA 模式，RGB565 格式为例）。对于该模块，我们只关心两点：1，如何存储图像数据；2，如何读取图像数据。

（1）如何存储图像数据。

PZ-OV7670 摄像头模块存储图像数据的过程为：等待 OV7670 同步信号→FIFO 写指针复位→FIFO 写使能→等待第二个 OV7670 同步信号→FIFO 写禁止。通过以上 5 个步骤，我们就完成了 1 帧图像数据的存储。

（2）如何读取图像数据。

在存储完一帧图像以后，我们就可以开始读取图像数据了。读取过程为：FIFO 读指针复位→给 FIFO 读时钟（FIFO_RCLK）→读取第一个像素高字节→给 FIFO 读时钟→读取第一个像素低字节→给 FIFO 读时钟→读取第二个像素高字节→循环读取剩余像素→结束。

可以看出，PZ-OV7670 摄像头模块数据的读取也是十分简单，比如 QVGA 模式，RGB565 格式，我们总共循环读取 $320 \times 240 \times 2$ 次，就可以读取 1 帧图像数据，把这些数据写入 LCD 模块，我们就可以看到摄像头捕捉到的画面了。

OV7670 还可以对输出图像进行各种设置，详见模块资料《OV7670 software application note》文档，对 AL422B 的操作时序，请大家参考 AL422B 的数据手册。

了解了 OV7670 模块的数据存储和读取，我们就可以开始设计程序了，本实验我们用一个外部中断来捕捉帧同步信号（VSYNC），然后在中断里面启动 OV7670 模块的图像数据存储，等待下一次 VSHNC 信号到来，我们就关闭数据存储，然后一帧数据就存储完成了，在主函数里面就可以慢慢的将这一帧数据读出

来，放到 LCD 即可显示了，同时开始第二帧数据的存储，如此循环，实现摄像头功能。

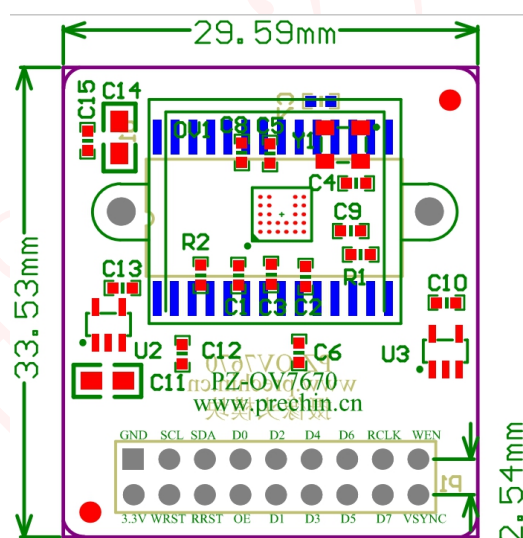
本实验我们将使用摄像头模块的 QVGA 输出（320*240），这个在我们使用的 TFTLCD 模块分辨率范围内。注意：PZ-0V7670 摄像头模块自带的 FIFO 是没办法缓存一帧的 VGA 图像的，如果使用 VGA 输出，那么你必须要在 FIFO 写满之前开始读 FIFO 数据，保证数据不被覆盖。

1.3 模块使用注意事项

在使用 PZ-0V7670 摄像头模块，当显示画面模糊时，可通过松开镜头座上面的螺丝后，旋转镜头来调节清晰度。

1.4 结构尺寸

PZ-0V7670 摄像头模块尺寸结构如下图所示：



2 硬件设计

2.1 硬件准备

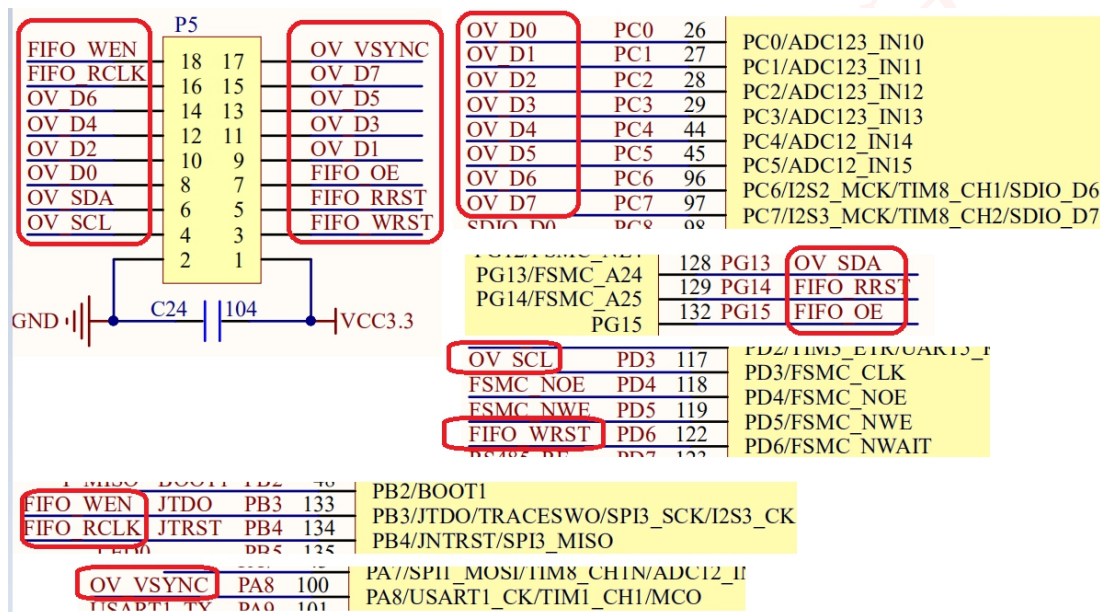
本实验所需要的硬件资源如下：

- ①玄武&朱雀开发板 1 个

- ②TFTLCD 模块
- ③PZ-OV7670 摄像头模块 1 个
- ④USB 线一条（用于供电和模块与电脑串口调试助手通信）

2.2 模块与开发板连接

PZ-OV7670 摄像头模块可直接与玄武&朱雀开发板板载的摄像头模块接口（P5）进行连接，该接口与 MCU 连接原理图（以玄武为例）如下图所示：



从上图可以看出，OV7670 摄像头模块的各信号脚与 STM32 的连接关系为：

- OV_SDA 接 PG13;
- OV_SCL 接 PD3;
- FIFO_RCLK 接 PB4;
- FIFO_WEN 接 PB3;
- FIFO_WRST 接 PD6;
- FIFO_RRST 接 PG14;
- FIFO_OE 接 PG15; OV_VSYNC 接 PA8;
- OV_D[7:0]接 PC[7:0];

这些线的连接，玄武&朱雀开发板的内部已经连接好了，我们只需要将 PZ-OV7670 摄像头模块插上去就好了。板载接口如下：



特别注意：模块插入开发板摄像头接口时注意方向，可对照板载接口背面 PCB 丝印电源脚位即可。

3 软件设计

本实验所实现的功能为：开机后，初始化摄像头模块（OV7670），如果初始化成功，则在 LCD 模块上面显示摄像头模块所拍摄到的内容。我们可以通过 K_UP 键设置光照模式（5 种模式）、通过 KEY1 键设置色饱和度，通过 KEY0 键设置亮度。通过串口我们可以查看当前的帧率（这里是指 LCD 显示的帧率，而不是指 OV7670 的输出帧率）。DS1 指示灯提示系统运行状态。

我们打开本实验工程，可以看到我们的工程 APP 列表中多了 ov7670.c 和 sccb.c 源文件，以及头文件 ov7670.h、sccb.h 和 ov7670cfg.h 等 5 个文件。本实验工程代码比较多，我们就不一一列出了，仅挑两个重要的地方进行讲解。

3.1 ov7670.c

首先，我们来看 ov7670.c 里面的 OV7670_Init 函数，该函数代码如下：

```
1. //初始化 OV7670
2. //返回 0:成功
3. //返回其他值:错误代码
4. u8 OV7670_Init(void)
5. {
6.     u8 temp;
7.     u16 i=0;
8.     //设置 IO
9.     GPIO_InitTypeDef GPIO_InitStructure;
10.    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA|RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOC|RCC_APB2Periph_GPIOD|RCC_APB2Periph_GPIOG|RCC_APB2Periph_AFIO, ENABLE); //使能相关端口时钟
11.
12.    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8; //PA8 输入 上拉
13.    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
14.    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
15.    GPIO_Init(GPIOA, &GPIO_InitStructure);
```

```

16.     GPIO_SetBits(GPIOA,GPIO_Pin_8);
17.
18.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3|GPIO_Pin_4;           // 端口配置
19.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;               //推挽输出
20.     GPIO_Init(GPIOB, &GPIO_InitStructure);
21.     GPIO_SetBits(GPIOB,GPIO_Pin_3|GPIO_Pin_4);
22.
23.     GPIO_InitStructure.GPIO_Pin = 0xff; //PC0~7 输入 上拉
24.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
25.     GPIO_Init(GPIOC, &GPIO_InitStructure);
26.
27.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
28.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
29.     GPIO_Init(GPIOD, &GPIO_InitStructure);
30.     GPIO_SetBits(GPIOD,GPIO_Pin_6);
31.
32.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_14|GPIO_Pin_15;
33.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
34.     GPIO_Init(GPIOG, &GPIO_InitStructure);
35.     GPIO_SetBits(GPIOG,GPIO_Pin_14|GPIO_Pin_15);
36.
37.     GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable,ENABLE); //SWD
38.
39.     SCCB_Init();           //初始化 SCCB 的 IO 口
40.     if(SCCB_WR_Reg(0x12,0x80))return 1; //复位 SCCB
41.     delay_ms(50);
42.     //读取产品型号
43.     temp=SCCB_RD_Reg(0x0b);
44.     if(temp!=0x73)return 2;
45.     temp=SCCB_RD_Reg(0x0a);
46.     if(temp!=0x76)return 2;
47.     //初始化序列
48.     for(i=0;i<sizeof(ov7670_init_reg_tbl)/sizeof(ov7670_init_reg_tbl[0]);i++)
49.     {
50.         SCCB_WR_Reg(ov7670_init_reg_tbl[i][0],ov7670_init_reg_tbl[i][1]);
51.     }
52.     return 0x00;           //ok
53. }

```

此部分代码先初始化 OV7670 相关的 IO 口（包括 SCCB_Init），然后最主要的是完成 OV7670 的寄存器序列初始化。OV7670 的寄存器特别多（上百来个），配置非常麻烦，幸好厂家有提供参考配置序列（详见《OV7670 software application note》），本实验我们用到的配置序列，存放在 ov7670_init_reg_tbl

这个数组里面，该数组是一个 2 维数组，存储初始化序列寄存器及其对应的值，该数组存放在 ov7670cfg.h 里面。

3.2 ov7670cfg.h

接下来，我们看看 ov7670cfg.h 里面 ov7670_init_reg_tbl 的内容，ov7670cfg.h 文件的代码如下：

```
1.  #ifndef _OV7670CFG_H
2.  #define _OV7670CFG_H
3.  #include "ov7670.h"
4.
5.
6.  //初始化寄存器序列及其对应的值
7.  const u8 ov7670_init_reg_tbl[][2]=
8.  {
9.      /*以下为 OV7670 QVGA RGB565 参数 */
10.     {0x3a, 0x04},//dummy
11.     {0x40, 0xd0},//565
12.     {0x12, 0x14},//QVGA,RGB 输出
13.
14.     //输出窗口设置
15.     {0x32, 0x80},//HREF control bit[2:0] HREF start 3 LSB    bit[5:3] HSTOP HREF end 3LSB
16.     {0x17, 0x16},//HSTART start high 8-bit MSB
17.     {0x18, 0x04},//5 HSTOP end high 8-bit
18.     {0x19, 0x02},
19.     {0x1a, 0x7b},//0x7a,
20.     {0x03, 0x06},//0x0a,帧垂直方向控制
21.
22.     {0x0c, 0x00},
23.     {0x15, 0x00},//0x00
24.     {0x3e, 0x00},//10
25.     {0x70, 0x3a},
26.     {0x71, 0x35},
27.     {0x72, 0x11},
28.     {0x73, 0x00},//
29.
30.     {0xa2, 0x02},//15
31.     {0x11, 0x81},//时钟分频设置,0,不分频.
32.     {0x7a, 0x20},
33.     {0x7b, 0x1c},
34.     {0x7c, 0x28},
35.
```

```

36.      {0x7d, 0x3c},//20
37.      {0x7e, 0x55},
38.      {0x7f, 0x68},
39.      {0x80, 0x76},
40.      {0x81, 0x80},
41.
42.      {0x82, 0x88},
43.      {0x83, 0x8f},
44.      {0x84, 0x96},
45.      {0x85, 0xa3},
46.      {0x86, 0xaf},
47.
48.      {0x87, 0xc4},//30
49.      {0x88, 0xd7},
50.      {0x89, 0xe8},
51.      {0x13, 0xe0},
52.      {0x00, 0x00},//AGC
53.
54.      {0x10, 0x00},
55.      {0x0d, 0x00},//全窗口, 位[5:4]: 01 半窗口, 10 1/4 窗口, 11 1/4 窗口
56.      {0x14, 0x28},//0x38, limit the max gain
57.      {0xa5, 0x05},
58.      {0xab, 0x07},
59.
60.      {0x24, 0x75},//40
61.      {0x25, 0x63},
62.      {0x26, 0xA5},
63.      {0x9f, 0x78},
64.      {0xa0, 0x68},
65.
66.      {0xa1, 0x03},//0x0b,
67.      {0xa6, 0xdf},//0xd8,
68.      {0xa7, 0xdf},//0xd8,
69.      {0xa8, 0xf0},
70.      {0xa9, 0x90},
71.
72.      {0xaa, 0x94},//50
73.      {0x13, 0xe5},
74.      {0x0e, 0x61},
75.      {0x0f, 0x4b},
76.      {0x16, 0x02},
77.
78.      {0x1e, 0x27},//图像输出镜像控制.0x07
79.      {0x21, 0x02},

```

```

80.      {0x22, 0x91},
81.      {0x29, 0x07},
82.      {0x33, 0x0b},
83.
84.      {0x35, 0x0b}, //60
85.      {0x37, 0x1d},
86.      {0x38, 0x71},
87.      {0x39, 0x2a},
88.      {0x3c, 0x78},
89.
90.      {0x4d, 0x40},
91.      {0x4e, 0x20},
92.      {0x69, 0x00},
93.      {0x6b, 0x40}, //PLL*4=48Mhz
94.      {0x74, 0x19},
95.      {0x8d, 0x4f},
96.
97.      {0x8e, 0x00}, //70
98.      {0x8f, 0x00},
99.      {0x90, 0x00},
100.     {0x91, 0x00},
101.     {0x92, 0x00}, //0x19, //0x66
102.
103.     {0x96, 0x00},
104.     {0x9a, 0x80},
105.     {0xb0, 0x84},
106.     {0xb1, 0x0c},
107.     {0xb2, 0x0e},
108.
109.     {0xb3, 0x82}, //80
110.     {0xb8, 0x0a},
111.     {0x43, 0x14},
112.     {0x44, 0xf0},
113.     {0x45, 0x34},
114.
115.     {0x46, 0x58},
116.     {0x47, 0x28},
117.     {0x48, 0x3a},
118.     {0x59, 0x88},
119.     {0x5a, 0x88},
120.
121.     {0x5b, 0x44}, //90
122.     {0x5c, 0x67},
123.     {0x5d, 0x49},

```



```

124.    {0x5e, 0x0e},
125.    {0x64, 0x04},
126.    {0x65, 0x20},
127.
128.    {0x66, 0x05},
129.    {0x94, 0x04},
130.    {0x95, 0x08},
131.    {0x6c, 0x0a},
132.    {0x6d, 0x55},
133.
134.
135.    {0x4f, 0x80},
136.    {0x50, 0x80},
137.    {0x51, 0x00},
138.    {0x52, 0x22},
139.    {0x53, 0x5e},
140.    {0x54, 0x80},
141.
142.    //{0x54, 0x40},//110
143.
144.
145.    {0x09, 0x03},//驱动能力最大
146.
147.    {0x6e, 0x11},//100
148.    {0x6f, 0x9f},//0x9e for advance AWB
149.    {0x55, 0x00},//亮度
150.    {0x56, 0x40},//对比度 0x40
151.    {0x57, 0x40},//0x40, change according to Jim's request
152.    //////////////////////////////////////
153.    //以下部分代码由开源电子网网友:duanzhang512 提出
154.    //添加此部分代码将可以获得更好的成像效果,但是最下面一行会有蓝色的抖动.
155.    //如不想要,可以屏蔽此部分代码.然后将:OV7670_Window_Set(12,176,240,320);
156.    //改为:OV7670_Window_Set(12,174,240,320);,即可去掉最下一行的蓝色抖动
157.    {0x6a, 0x40},
158.    {0x01, 0x40},
159.    {0x02, 0x40},
160.    {0x13, 0xe7},
161.    {0x15, 0x00},
162.
163.
164.    {0x58, 0x9e},
165.
166.    {0x41, 0x08},
167.    {0x3f, 0x00},

```

168.	{0x75, 0x05},
169.	{0x76, 0xe1},
170.	{0x4c, 0x00},
171.	{0x77, 0x01},
172.	{0x3d, 0xc2},
173.	{0x4b, 0x09},
174.	{0xc9, 0x60},
175.	{0x41, 0x38},
176.	
177.	{0x34, 0x11},
178.	{0x3b, 0x02},
179.	
180.	{0xa4, 0x89},
181.	{0x96, 0x00},
182.	{0x97, 0x30},
183.	{0x98, 0x20},
184.	{0x99, 0x30},
185.	{0x9a, 0x84},
186.	{0x9b, 0x29},
187.	{0x9c, 0x03},
188.	{0x9d, 0x4c},
189.	{0x9e, 0x3f},
190.	{0x78, 0x04},
191.	
192.	{0x79, 0x01},
193.	{0xc8, 0xf0},
194.	{0x79, 0x0f},
195.	{0xc8, 0x00},
196.	{0x79, 0x10},
197.	{0xc8, 0x7e},
198.	{0x79, 0x0a},
199.	{0xc8, 0x80},
200.	{0x79, 0x0b},
201.	{0xc8, 0x01},
202.	{0x79, 0x0c},
203.	{0xc8, 0x0f},
204.	{0x79, 0x0d},
205.	{0xc8, 0x20},
206.	{0x79, 0x09},
207.	{0xc8, 0x80},
208.	{0x79, 0x02},
209.	{0xc8, 0xc0},
210.	{0x79, 0x03},
211.	{0xc8, 0x40},

```

212.     {0x79, 0x05},
213.     {0xc8, 0x30},
214.     {0x79, 0x26},
215.     {0x09, 0x00},
216.     //////////////////////////////////////
217.
218. };
219.
220. #endif

```

以上代码，我们大概了解下结构，每个条目的第一个字节为寄存器号（也就是寄存器地址），第二个字节为要设置的值，比如 {0x3a, 0x04}，就表示在 0X03 地址，写入 0X04 这个值。

通过这么一长串（110 多个）寄存器的配置，我们就完成了 OV7670 的初始化，本实验我们配置 OV7670 工作在 QVGA 模式，RGB565 格式输出。在完成初始化之后，我们既可以开始读取 OV7670 的数据了。

OV7670 文件夹里面的其他代码我们就不逐个介绍了，请大家参考该例程源码。

3.3 time.c

因为本实验我们还用到了帧率（LCD 显示的帧率）统计和中断处理，所以我们还需要修改 time.c、time.h 这几个文件。

在 time.c 里面，我们使用 TIM4 用于统计帧率，其代码如下：

```

1.  /*****
2.  * 函数名      : TIM4_Init
3.  * 函数功能    : TIM4 初始化函数
4.  * 输 入      : per:重装载值
5.                psc:分频系数
6.  * 输 出      : 无
7.  *****/
8.  void TIM4_Init(u16 per,u16 psc)
9.  {
10.     TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
11.     NVIC_InitTypeDef NVIC_InitStructure;
12.
13.     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4,ENABLE); //使能 TIM4 时钟
14.
15.     TIM_TimeBaseInitStructure.TIM_Period=per; //自动装载值

```

```

16.     TIM_TimeBaseInitStructure.TIM_Prescaler=psc; //分频系数
17.     TIM_TimeBaseInitStructure.TIM_ClockDivision=TIM_CKD_DIV1;
18.     TIM_TimeBaseInitStructure.TIM_CounterMode=TIM_CounterMode_Up; //设置向上计数模式
19.     TIM_TimeBaseInit(TIM4,&TIM_TimeBaseInitStructure);
20.
21.     TIM_ITConfig(TIM4,TIM_IT_Update,ENABLE); //开启定时器中断
22.     TIM_ClearITPendingBit(TIM4,TIM_IT_Update);
23.
24.     NVIC_InitStructure.NVIC_IRQChannel = TIM4_IRQn;//定时器中断通道
25.     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority=2;//抢占优先级
26.     NVIC_InitStructure.NVIC_IRQChannelSubPriority =3;          //子优先级
27.     NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;           //IRQ 通道使能
28.     NVIC_Init(&NVIC_InitStructure);
29.
30.     TIM_Cmd(TIM4,ENABLE); //使能定时器
31. }
32.
33. u8 ov_frame;    //统计帧数
34. /*****
35.  * 函 数 名      : TIM4_IRQHandler
36.  * 函数功能      : TIM4 中断函数
37.  * 输    入      : 无
38.  * 输    出      : 无
39.  *****/
40. void TIM4_IRQHandler(void)
41. {
42.     if(TIM_GetITStatus(TIM4,TIM_IT_Update))
43.     {
44.         LED2=!LED2;
45.         printf("frame:%dfps\r\n",ov_frame); //打印帧率
46.         ov_frame=0;
47.     }
48.     TIM_ClearITPendingBit(TIM4,TIM_IT_Update);
49. }

```

这里我们用到通用定时器 TIM4 来统计帧率，也就是 1 秒钟中断一次，打印 ov_frame 的值， ov_frame 用于统计 LCD 帧率。

在 ov7670.c 里面添加 EXTI8_Init 和 EXTI9_5_IRQHandler 函数，用于 OV7670 模块的 FIFO 写控制， ov7670.c 文件新增部分代码如下：

```

1. //外部中断 8 初始化
2. void EXTI8_Init(void)
3. {
4.     EXTI_InitTypeDef EXTI_InitStructure;

```

```

5.     NVIC_InitTypeDef NVIC_InitStructure;
6.
7.     GPIO_EXTILineConfig(GPIO_PortSourceGPIOA,GPIO_PinSource8);//PA8 对中断线 8
8.
9.     EXTI_InitStructure.EXTI_Line=EXTI_Line8;
10.    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
11.    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Rising;
12.    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
13.    EXTI_Init(&EXTI_InitStructure);    //根据 EXTI_InitStruct 中指定的参数初始化外设 EXTI 寄存
    器
14.
15.    NVIC_InitStructure.NVIC_IRQChannel = EXTI9_5_IRQn;    //使能按键所在的外部中断通道
16.    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;    //抢占优先级 0
17.    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;    //子优先级 0
18.    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;    //使能外部中
    断通道
19.    NVIC_Init(&NVIC_InitStructure);    //根据 NVIC_InitStruct 中指定的参数初始化外设 NVIC 寄存
    器
20. }
21.
22. u8 ov_sta; //帧中断标记
23. void EXTI9_5_IRQHandler(void)    //红外遥控外部中断
24. {
25.     if(EXTI_GetITStatus(EXTI_Line8)==SET)    //是 8 线的中断
26.     {
27.         if(ov_sta<2)
28.         {
29.             if(ov_sta==0)
30.             {
31.                 OV7670_WRST=0; //复位写指针
32.                 OV7670_WRST=1;
33.                 OV7670_WREN=1; //允许写入 FIFO
34.             }
35.             else
36.             {
37.                 OV7670_WREN=0; //禁止写入 FIFO
38.                 OV7670_WRST=0; //复位写指针
39.                 OV7670_WRST=1;
40.             }
41.             ov_sta++;    //帧中断加 1
42.         }
43.     }
44.     EXTI_ClearITPendingBit(EXTI_Line8); //清除 EXTI8 线路挂起位
45. }

```

因为 OV7670 的帧同步信号（OV_VSYNC）接在 PA8 上面，所以我们这里配置 PA8 作为中断输入，因为 STM32 的外部中断 5~9 共用一个中断服务函数（EXTI9_5_IRQHandler），所以在该函数里面，我们需要先判断中断是不是来自中断线 8 的，然后再做处理。

中断处理部分很简单，通过一个 ov_sta 来控制 OV7670 模块的 FIFO 写操作。当 ov_sta=0 的时候，表示 FIFO 存储的数据已经被成功读取了（ov_sta 在读完 FIFO 数据的时候被清零），然后只要 OV_VSYNC 信号到来，我们就先复位一下写指针，然后 ov_sta=1，标志着写指针已经复位，目前正在往 FIFO 里面写数据。再等下一个 OV_VSYNC 到来，也就表明一帧数据已经存储完毕了，此时我们设置 OV7670_WREN 为 0，禁止再往 OV7670 写入数据，此时 ov_sta 自增为 2。其他程序，只要读到 ov_sta 为 2，就表示一帧数据已经准备好了，可以读出，在读完数据之后，程序设置 ov_sta 为 0，则开启下一轮 FIFO 数据存储。

3.4 main.c

最后我们看下 main.c 文件，代码如下：

```
1.  #include "system.h"
2.  #include "SysTick.h"
3.  #include "led.h"
4.  #include "usart.h"
5.  #include "tftlcd.h"
6.  #include "key.h"
7.  #include "malloc.h"
8.  #include "sd_sdio.h"
9.  #include "flash.h"
10. #include "ff.h"
11. #include "fatfs_app.h"
12. #include "font_show.h"
13. #include "ov7670.h"
14. #include "time.h"
15. #include "string.h"
16. #include "math.h"
17.
18.
19. extern u8 ov_sta;
20. extern u8 ov_frame; //在 time.c 里面定义
21.
```

```

22. //更新 LCD 显示
23. void camera_refresh(void)
24. {
25.     u32 j;
26.     u16 color;
27.     if(ov_sta)//有帧中断更新?
28.     {
29.         LCD_Display_Dir(1);
30.         LCD_Set_Window(0,0,320-1,240-1);//将显示区域设置到屏幕中央
31.
32.         OV7670_RRST=0;           //开始复位读指针
33.         OV7670_RCK_L;
34.         OV7670_RCK_H;
35.         OV7670_RCK_L;
36.         OV7670_RRST=1;           //复位读指针结束
37.         OV7670_RCK_H;
38.
39.         for(j=0;j<76800;j++) //此种方式需清楚 TFT 内部显示方向控制寄存器值 速度较快
40.         {
41.             OV7670_RCK_L;
42.             color=GPIOC->IDR&0XFF; //读数据
43.             OV7670_RCK_H;
44.             color<<=8;
45.             OV7670_RCK_L;
46.             color|=GPIOC->IDR&0XFF; //读数据
47.             OV7670_RCK_H;
48.             LCD_WriteData_Color(color);
49.         }
50.         ov_sta=0;           //清零帧中断标记
51.         ov_frame++;
52.         LCD_Display_Dir(0);
53.     }
54. }
55.
56. const u8*LMODE_TBL[5]={"Auto", "Sunny", "Cloudy", "Office", "Home"};
57. const u8*EFFECTS_TBL[7]={"Normal", "Negative", "B&W", "Redish", "Greenish", "Bluish", "Antique"};
    //7 种特效
58.
59.
60. int main()
61. {
62.     u8 i=0;
63.     u8 key;
64.     u8 lightmode=0,saturation=2,brightness=2,contrast=2;

```

```

65.     u8 effect=0;
66.     u8 sbuf[15];
67.     u8 count;
68.
69.     SysTick_Init(72);
70.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
71.     LED_Init();
72.     USART1_Init(115200);
73.     TFTLCD_Init(); //LCD 初始化
74.     KEY_Init();
75.     EN25QXX_Init(); //初始化 EN25Q128
76.     my_mem_init(SRAMIN); //初始化内部内存池
77.
78.     FRONT_COLOR=RED; //设置字体为红色
79.     LCD_ShowString(10,10,tftlcd_data.width,tftlcd_data.height,16,"PRECHIN STM32F1");
80.     LCD_ShowString(10,30,tftlcd_data.width,tftlcd_data.height,16,"www.prechin.net");
81.     LCD_ShowString(10,50,tftlcd_data.width,tftlcd_data.height,16,"OV7670 Test");
82.
83.     while(OV7670_Init())//初始化 OV7670
84.     {
85.         LCD_ShowString(10,80,tftlcd_data.width,tftlcd_data.height,16,"OV7670 Error!");
86.         delay_ms(200);
87.         LCD_Fill(10,80,239,206,WHITE);
88.         delay_ms(200);
89.     }
90.     LCD_ShowString(10,80,tftlcd_data.width,tftlcd_data.height,16,"OV7670 OK! ");
91.     delay_ms(1500);
92.     OV7670_Light_Mode(0);
93.     OV7670_Color_Saturation(2);
94.     OV7670_Brightness(2);
95.     OV7670_Contrast(2);
96.     OV7670_Special_Effects(0);
97.
98.     TIM4_Init(10000,7199); //10Khz 计数频率,1秒钟中
    断
99.     EXTI8_Init();
100.    OV7670_Window_Set(12,176,240,320); //设置窗口
101.    OV7670_CS=0;
102.    LCD_Clear(BLACK);
103.    while(1)
104.    {
105.        key=KEY_Scan(0);
106.        if(key)count=20;
107.        switch(key)

```



```

108.     {
109.         case KEY_UP_PRESS:           //灯光模式设置
110.             lightmode++;
111.             if(lightmode>4)lightmode=0;
112.             OV7670_Light_Mode(lightmode);
113.             sprintf((char*)sbuf, "%s", LMODE_TBL[lightmode]);
114.             break;
115.         case KEY1_PRESS:              //饱和度
116.             saturation++;
117.             if(saturation>4)saturation=0;
118.             OV7670_Color_Saturation(saturation);
119.             sprintf((char*)sbuf, "Saturation:%d", (char)saturation-2);
120.             break;
121.         case KEY0_PRESS:              //对比度
122.             contrast++;
123.             if(contrast>4)contrast=0;
124.             OV7670_Contrast(contrast);
125.             sprintf((char*)sbuf, "Contrast:%d", (char)contrast-2);
126.             break;
127.     }
128.     if(count)
129.     {
130.         count--;
131.         LCD_ShowString((tftlcd_data.width-240)/2+30, (tftlcd_data.height-320)/2+60, 200, 1
            6, 16, sbuf);
132.     }
133.     camera_refresh();//更新显示
134.     i++;
135.     if(i%20==0)
136.     {
137.         LED1=!LED1;
138.     }
139.     // delay_ms(5);
140. }
141. }

```

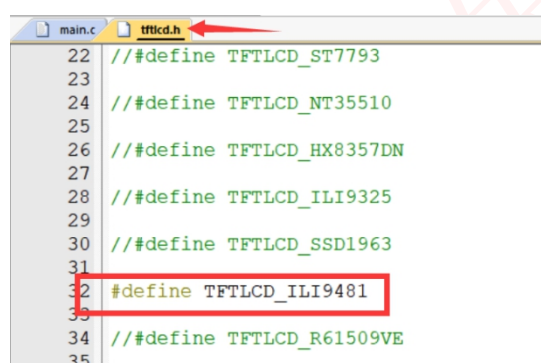
此部分代码除了 mian 函数，还有一个 camera_refresh 函数，该函数用于将摄像头模块 FIFO 的数据读出，并显示在 LCD 上面。main 函数则比较简单，我们就不细说了。

4 实验现象

首先，请先确保硬件都已经连接好：

- ①PZ-OV7670 摄像头模块与玄武&朱雀开发板连接（可参考硬件设计小节）
- ②开发板插上 TFTLCD 液晶。
- ③使用 USB 线给开发板供电。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：

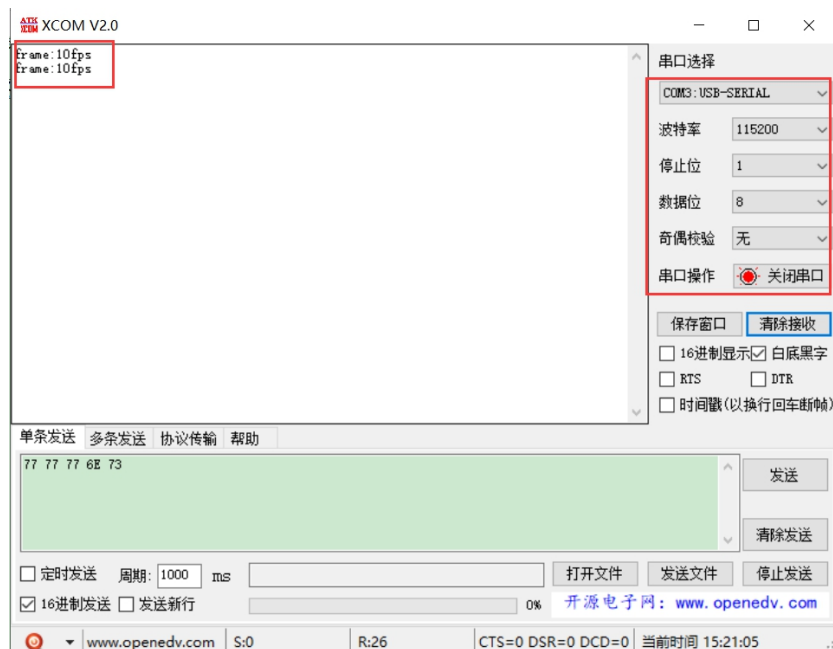


```
22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以玄武开发板为展示）可以看到 TFTLCD 上显示摄像头拍摄的画面，LCD 界面显示如下图所示：



此时，我们可以按不同的按键（KEY_UP、KEY1、KEY0），来设置摄像头的相关参数和模式，得到不同的成像效果。还可以通过串口打印输出 LCD 显示的帧率，如图所示：



从上图还可以看出，LCD 显示帧率为 10 帧，则可以推断 OV7670 的输出帧率则是 $3 \times 10 = 30$ 帧。当然对于不同 LCD 驱动帧率不一样，有的可能输出 8 帧左右，那么 OV7670 的输出帧率至少是在 24 帧以上，实际输出为 30 帧。

5 其他

(1) 购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

(2) 资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

(3) 技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）